

PROJECT AND TEAM INFORMATION

Project Title

ClashFree: An OOP-Based Timetable Scheduler Using Graph Coloring and Constraint Satisfaction

Student / Team Information

<i>Team ID: DSCPP-III-2026-T052</i> <i>Mentor: (Prof.) Dr.Deepak Asrani (deepakasrani.cse@geu.ac.in)</i> <i>Team:</i>	<i>Details</i>
Team member 1 (Team Lead) <i>(Last Name, name: student ID: email, picture):</i>	<i>Ahuja, Nishchal –</i> <i>2510380045</i> <i>nishchalahuja28@gmail.com</i> 
Team member 2 <i>(Last Name, name: student ID: email, picture):</i>	<i>Bhatt, Bipul – 2510370355</i> <i>bipul88bhatt08@gmail.com</i> 
Team member 3 <i>(Last Name, name: student ID: email, picture):</i>	<i>Singh, Mohit – 2510370453</i> <i>mohit.singh.bh6@gmail.com</i> 
Team member 4 <i>(Last Name, name: student ID: email, picture):</i>	<i>Tadiyal, Prrajval –</i> <i>2510360034</i> <i>prrajvaltadiyal@gmail.com</i> 

PROPOSAL DESCRIPTION

Motivation

- **High Computational Combinatorial Complexity:** Academic timetabling in modern educational institutions is a highly complex combinatorial optimization task known mathematically as the University Course Timetabling Problem (UCTP), which is proven to be NP-hard.
- **Vulnerability of Manual Approaches:** Traditional spreadsheet-based timetabling relies heavily on manual coordination among administrative staff, making it prone to critical hard-constraint violations such as assigning the same instructor to multiple concurrent classes, overbooking room physical capacities, or creating section overlap clashes.
- **Neglect of Soft-Constraint Quality:** Manual scheduling methods typically struggle to optimize soft constraints, resulting in severely fragmented student timetables with long idle gaps between sessions, unbalanced daily teaching loads for faculty members, and excessive room transitions across campus.
- **Institutional Scalability Bottlenecks:** As institutional enrolments grow, elective options diversify, and multi-departmental course offerings expand, manual coordination becomes computationally intractable, resource-intensive, and prone to human oversight.
- **Necessity of an Automated Algorithmic Solution:** Developing an automated, object-oriented software system provides a mathematically formal algorithmic framework that guarantees hard-constraint satisfaction, optimizes resource utilization, minimizes schedule fragmentation, and drastically reduces administrative preparation overhead.

State of the Art / Current solution

- **Manual Spreadsheet-Based Planning:** Most academic departments rely primarily on manual spreadsheet software (e.g., Microsoft Excel, Google Sheets) paired with ad-hoc human review. This approach lacks real-time conflict detection, automated optimization capabilities, and computational scalability.
- **Commercial Off-The-Shelf (COTS) ERP Systems:** Enterprise academic management systems exist, but they operate as expensive, closed-source “black box” platforms. They offer limited flexibility for custom institutional rules (e.g., contiguous multi-slot practical lab allocations) and lack runtime transparency.
- **Absence of Benchmarking Frameworks:** Existing institutional setups rarely provide research-grade visibility or modular frameworks to evaluate, compare, and benchmark graph-coloring dynamic heuristics against exact constraint satisfaction algorithms.

Project Goals and Milestones

Core Project Goal: Design, implement, and evaluate an efficient, object-oriented C++ software engine that models institutional timetabling as a Constraint Satisfaction Problem (CSP) via graph coloring, integrating customizable heuristic strategies and soft-constraint local search optimization engines. The project will be evaluated through three project phases (TA1-TA3) and the End Semester Theory Examination (TA4).

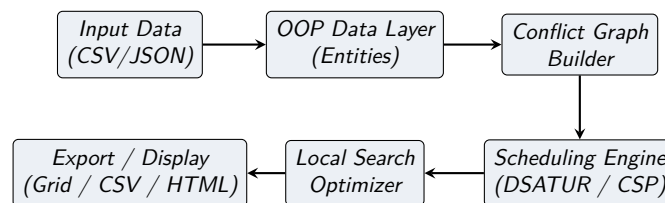
- **TA1 (Project Proposal Evaluation):** Formulate the project title, problem statement, core technical objectives, proposed object-oriented C++ architecture, curriculum subject integration, and deliver initial domain model progress.
- **TA2 (Mid-Term Progress Evaluation):** Build and evaluate the functional scheduling engine core, assess working code implementation, review technical progress and subject integration, conduct regular mentor interactions, and track individual contributions.
- **TA3 (Final Project Evaluation):** Deliver a complete working project, execute a live system demonstration, submit technical reports, demonstrate overall technical mastery, present individual contributions, and clear the oral viva voce examination.
- **TA4 (End Semester Theory Examination):** Comprehensive written theoretical assessment

testing domain knowledge in C++ Object-Oriented Programming, Data Structures, Graph Algorithms, and Constraint Satisfaction techniques.

Project Approach

- **Mathematical Problem Formulation:** The timetabling task is mapped onto a classic Vertex Graph Coloring model $G = (V, E)$. Each vertex $v \in V$ represents a unique (course, section) lecture or lab session. An undirected edge $e = (u, v) \in E$ denotes a hard conflict (shared faculty member, room, or student cohort). Discrete time slots act as colors $c \in \{1, \dots, K\}$.
- **Object-Oriented Architecture Software Patterns:** The system is designed following strict object-oriented principles. The `SchedulingStrategy` base interface uses the Strategy Design Pattern to decouple the scheduling logic, allowing seamless switching between concrete implementations: `GreedyScheduler`, `BacktrackScheduler`, and `DSATURScheduler`.
- **Multi-Phase Solvers Optimization Pipeline:** The execution pipeline flows sequentially from hard-constraint resolution (graph coloring / CSP search) to soft-constraint minimization via a post-processing Local Search engine (minimizing faculty load variance and student idle gaps).
- **Technology Stack Language Standards:** Developed in modern standard C++20 using standard compilers (GCC/Clang) and STL container data structures (`std::vector`, `std::unordered_map`, `std::priority_queue`) to maximize computational speed, ensure fine-grained memory management, and eliminate external heavyweight framework dependencies.

System Architecture (High Level Diagram)



- **Input Data Ingestion:** Ingests institutional CSV/JSON configuration files defining entities, availability, and resource constraints.
- **Domain Modeling Layer:** Instantiates C++ domain objects (`Course`, `Faculty`, `Room`, `Section`) enforcing structural integrity.
- **Conflict Graph Construction:** Maps non-shareable resource overlaps into an adjacency-list undirected conflict graph.
- **Core Scheduling Engine:** Applies strategy-based graph coloring heuristics (Greedy, Backtracking, DSATUR) to enforce hard constraints.
- **Post-Processing Optimizer:** Refines schedule quality using local search algorithms to minimize soft-constraint penalties.
- **Export Display Exporter:** Renders clean schedule views in CLI tabular grids, CSV export files, and interactive HTML dashboards.

Project Outcome / Deliverables

- **Production C++ Scheduling Application:** A fully compiled, modular C++ application implementing Greedy, Backtracking, and DSATUR graph coloring algorithms using the Strategy Pattern.
- **Empirical Benchmarking Profiling Suite:** Comprehensive benchmarking tools to evaluate dynamic saturation metrics, algorithmic runtimes, chromatic numbers, and soft-constraint violation penalties.
- **Multi-Format Schedule Exporters:** CLI terminal matrix renderer along with automated CSV and HTML report visualization generators for institutional deployment.
- **Academic Technical Documentation:** Comprehensive final report covering NP-hardness reduction proofs, heuristic algorithmic complexity analysis, and empirical performance evaluations.

Assumptions

- **Fixed Temporal Grid:** Operating schedule runs on a standard weekly grid structure consisting of 5 working days with 8 periods per day (40 total discrete time slots).
- **Static Resource Inputs:** Classroom capacities, faculty availability matrices, and course enrollment attributes are static and provided upfront.
- **Continuous Practical Block Allocation:** Laboratory and practical sessions require uninterrupted, continuous multi-slot allocations on the same day.
- **Negligible Travel Overhead:** Transit times between classrooms within the same campus block are assumed to be negligible between consecutive class periods.

References

1. Brélaz, D. (1979). New methods to color the vertices of a graph. *Communications of the ACM*, 22(4), 251–256.
2. Lewis, R. (2016). *A Guide to Graph Colouring: Algorithms and Applications*. Springer International Publishing.
3. Burke, E. K., & Petrovic, S. (2002). Recent research directions in automated timetabling. *European Journal of Operational Research*, 140(2), 266–280.